



Towards Understanding Fine-Grained Programming Mistakes and Fixing Patterns in Data Science

Wei-Hao Chen, Jia Lin Cheoh, Manthan Keim, Sabine Brunswicker, and Tianyi Zhang

Data Science Programming

- Computational notebooks
- Cells are executed in non-linear orders
- Limited debugging tools



Markdown Cell — Create a line $y = 2x + b$ where x is gaussian distributed and b is uniformly distributed

Code Cell —

```
import numpy as np
x = np.random.normal(0, 1, 100)
b = np.random.uniform(0, 1, 100)
y = 2 * x + b
print(np.corrcoef(x, y))
```

[1] ✓ 0.0s Python

Output —

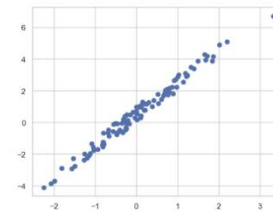
```
[[1.          0.9826778]
 [0.9826778  1.          ]]
```

Draw a scatter plot of the points for $y = 2x + b$

Index — [3]

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
sns.set_style("whitegrid")
plt.scatter(x, y)
```

✓ 0.0s Python



An example of a notebook

Data Science Programming

- Many variables and function in different cells
- Non-linear execution orders
- Cell dependency issues



Subotić et al., “A static analysis framework for data science notebooks.” ICSE-SEIP 2022

Chattopadhyay et al., “What's wrong with computational notebooks? Pain points, needs, and design opportunities.” CHI 2020

Previous Work

- Previous work does not fully capture the **fine-grained** programming behavior.
- Notebooks from Kaggle or GitHub only contain **final version or committed version** of the notebooks.

Biswas et al., "The art and practice of data science pipelines: A comprehensive study of data science pipelines in theory, in-the-small, and in-the-large." ICSE 2022

Groto et al., "A large-scale comparison of Python code in Jupyter notebooks and scripts." MSR 2022.

Raghunandan et al., "Code code evolution: Understanding how people change data science notebooks over time." CHI 2023

Previous Work

- Previous work does not fully capture the **fine-grained** programming behavior.
- Notebooks from Kaggle or GitHub only contain **final version or committed version** of the notebooks.

Cell A

Cell B

Previous Work

- Previous work does not fully capture the **fine-grained** programming behavior.
- Notebooks from Kaggle or GitHub only contain **final version or committed version** of the notebooks.

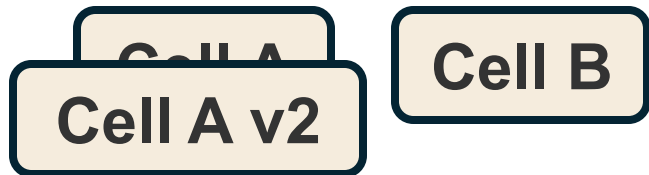
Cell A

Cell B

Execution History:
Run A → Run B

Previous Work

- Previous work does not fully capture the **fine-grained** programming behavior.
- Notebooks from Kaggle or GitHub only contain **final version or committed version** of the notebooks.

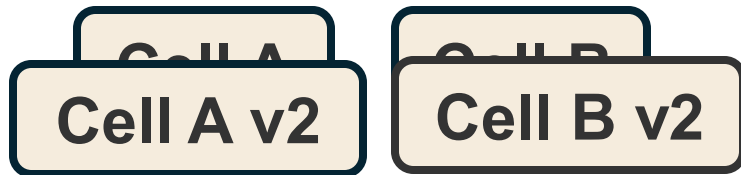


Execution History:

Run A → Run B → Edit A → Run A

Previous Work

- Previous work does not fully capture the **fine-grained** programming behavior.
- Notebooks from Kaggle or GitHub only contain **final version or committed version** of the notebooks.

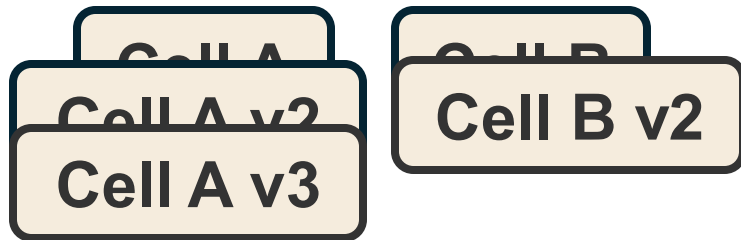


Execution History:

Run A → Run B → Edit A → Run A → Edit B → Run B

Previous Work

- Previous work does not fully capture the **fine-grained** programming behavior.
- Notebooks from Kaggle or GitHub only contain **final version or committed version** of the notebooks.

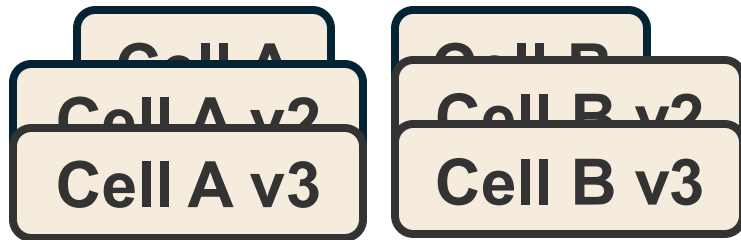


Execution History:

Run A → Run B → Edit A → Run A → Edit B → Run B → Edit A
→ Run A

Previous Work

- Previous work does not fully capture the **fine-grained** programming behavior.
- Notebooks from Kaggle or GitHub only contain **final version or committed version** of the notebooks.

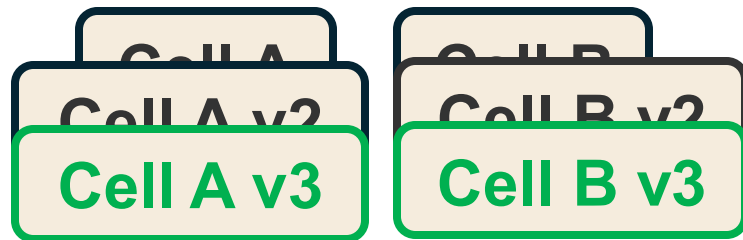


Execution History:

Run A → Run B → Edit A → Run A → Edit B → Run B → Edit A
→ Run A → Edit B → Run B

Previous Work

- Previous work does not fully capture the **fine-grained** programming behavior.
- Notebooks from Kaggle or GitHub only contain **final version or committed version** of the notebooks.



Committed Version

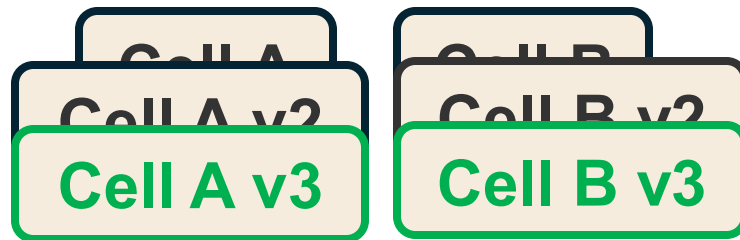


Execution History:

Run A → Run B → Edit A → Run A → Edit B → Run B → Edit A
→ Run A → Edit B → Run B

Previous Work

- Previous work does not fully capture the **fine-grained** programming behavior.
- Notebooks from Kaggle or GitHub only contain **final version or committed version** of the notebooks.



Committed Version



Execution History:

Missing!

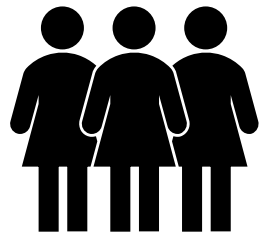
Run A → Run B → Edit A → Run A → Edit B → Run B → Edit A
→ Run A → Edit B → Run B

Research Questions

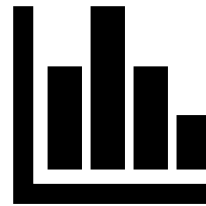
- Fine-grained programming mistakes?
- Debugging activities?
- Fix patterns applied to the errors?
- Reasons for these errors?

Dataset

6-week DS
competition with
67 participants



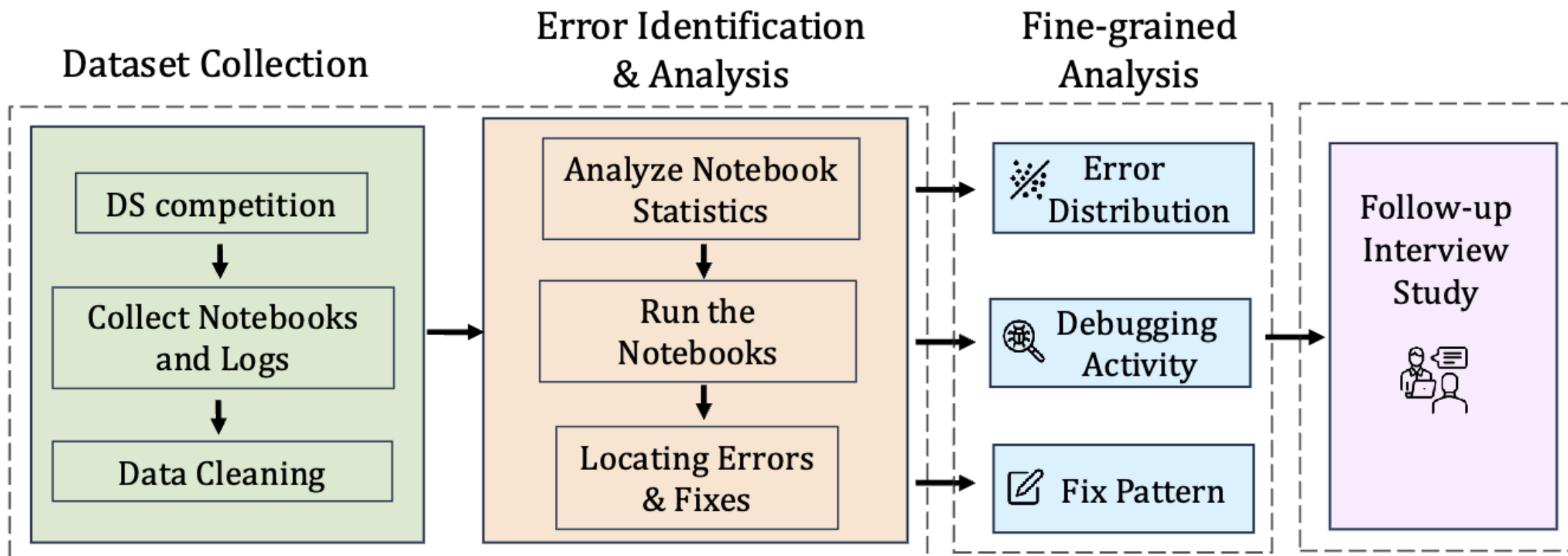
Collected 1,310
notebooks, sampled
390 for analysis



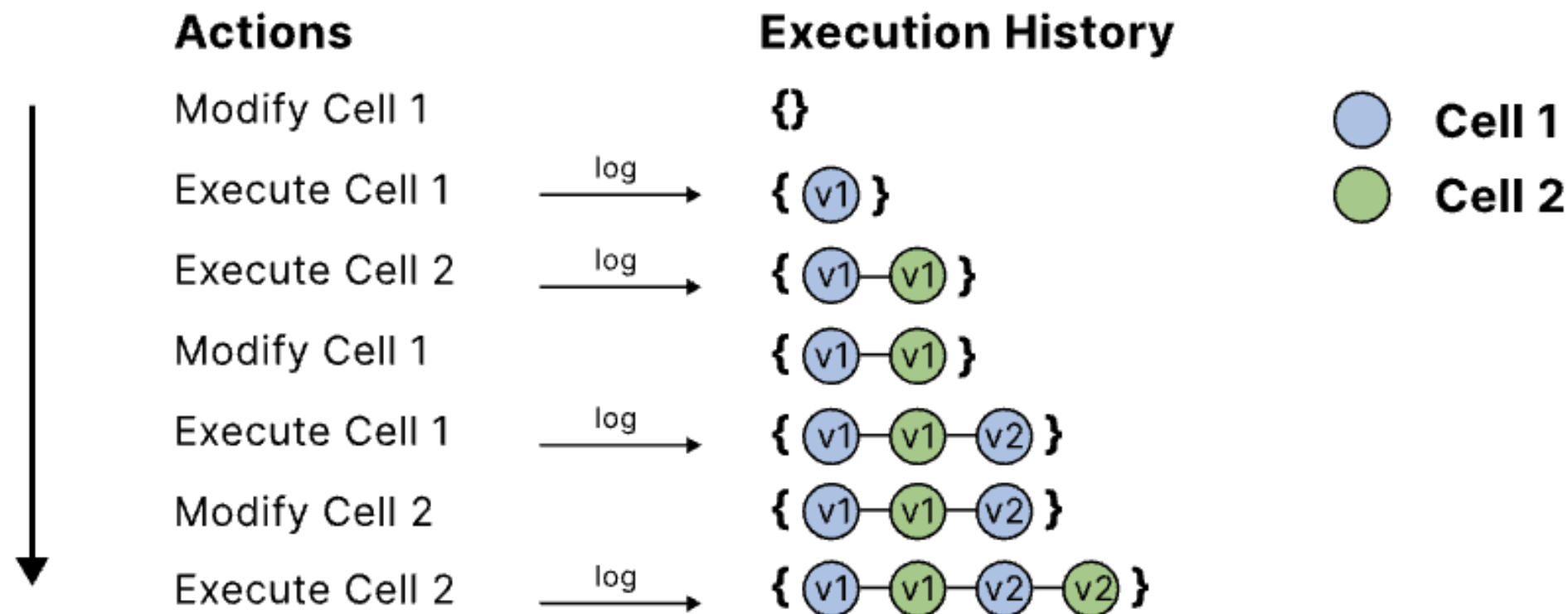
10 follow-up
interviews



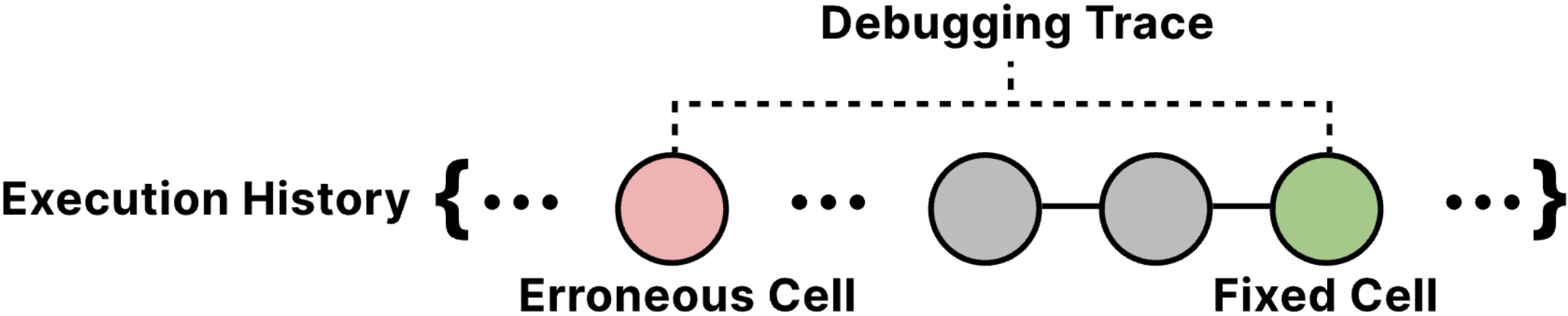
Analysis Procedure



Execution History



Debugging Trace



Debugging Activity

- **Rerun Previous Cell:**

- A previous cell, not the erroneous cell, has been re-executed without any modifications.

- **Rerun Erroneous Cell:**

- The original erroneous cell is re-executed without any changes.

- **Edit Erroneous Cell:**

- The programmer edited the erroneous cell.

- **Edit Previous Cell:**

- The programmer edited any previous cell, not the erroneous cell.

- **Create New Cell:**

- The programmer created a complete new cell.

Fix patterns

- 12 Fix patterns using GumTree predicates

EDIT PATTERN	INFERENCE RULE
Add New Method Call	$\text{Insert}(t_0, t_n, i) \wedge \text{NodeType}(t_n, \text{MethodCall})$
Add New Attribute	$\text{Insert}(t_0, t_n) \wedge \text{NodeType}(t_n, \text{Attribute})$
Change Parameters	$\text{Update}(t_0, t_n) \wedge (\text{NodeType}(t_n, \text{Parameters}) \vee \text{NodeType}(t_n, \text{ParameterValue})) \wedge (\text{NodeSet}(t_0) \neq \text{NodeSet}(t_n))$
Change Key/Index	$\text{Update}(t_0, t_n) \wedge (\text{NodeType}(t_n, \text{String}) \vee \text{NodeType}(t_n, \text{Number})) \wedge \text{IsInBrackets}(\text{NodeValue}(t_0, \text{oldVal}), \text{NodeValue}(t_n, \text{newVal}))$
Rename Variable	$\text{Rename}(t_0, t_n) \wedge \text{NodeType}(t_n, \text{Variable}) \wedge \text{NodeValue}(t_0) \neq \text{NodeValue}(t_n) \wedge (\text{NodeSet}(t_0) \neq \text{NodeSet}(t_n))$
Remove Method Call	$\text{Delete}(t_0) \wedge \text{NodeType}(t_0, \text{MethodCall})$
Remove Attribute	$\text{Delete}(t_0) \wedge \text{NodeType}(t_0, \text{Attribute})$
Delete Old Lines	$\text{Delete}(t_0) \wedge (\text{NumLines}(t_0) > \text{NumLines}(t_n))$
Add New Lines	$\text{Insert}(t_0, t_n, i) \wedge (\text{NumLines}(t_n) > \text{NumLines}(t_0))$
Commented Out Code	$\text{Update}(t_0, t_n) \wedge \text{NodeType}(t_n, \text{Comment}) \wedge \text{Contains}(\text{NodeValue}(t_0), \text{Trim}(\text{NodeValue}(t_n), "\#"))$
No Changes	$\text{Equal}(t_0, t_n)$
Fix Syntax Errors	$\text{SyntaxError}(t_0) \wedge \neg \text{SyntaxError}(t_n)$



Common Programming Errors

Stage	AttrErr	TypeErr	ValErr	NameErr	NotFnd	BadReq	KeyErr	SyntaxErr	IndexErr	ModNotFnd	Misc	Total
Data Loading	9	3	0	25	28	27	6	2	1	0	0	101
Data Preprocessing	25	17	32	25	0	0	25	12	4	0	0	140
Data Exploration	24	18	37	31	0	0	20	22	14	0	4	170
Modeling	1	0	8	3	0	0	1	1	0	0	1	15
Prediction	0	1	2	1	0	0	0	1	0	0	2	7
Evaluation	2	0	0	1	0	0	0	0	0	0	0	3
Visualization	9	5	13	16	0	0	3	8	0	0	2	56
Result Saving	0	0	0	0	0	0	0	0	0	0	0	0
Comment Only	0	0	0	0	0	0	0	0	0	0	0	0
Helper Functions	0	0	19	1	0	0	0	4	0	12	1	37
Total	70	44	111	103	28	27	55	50	19	12	10	529

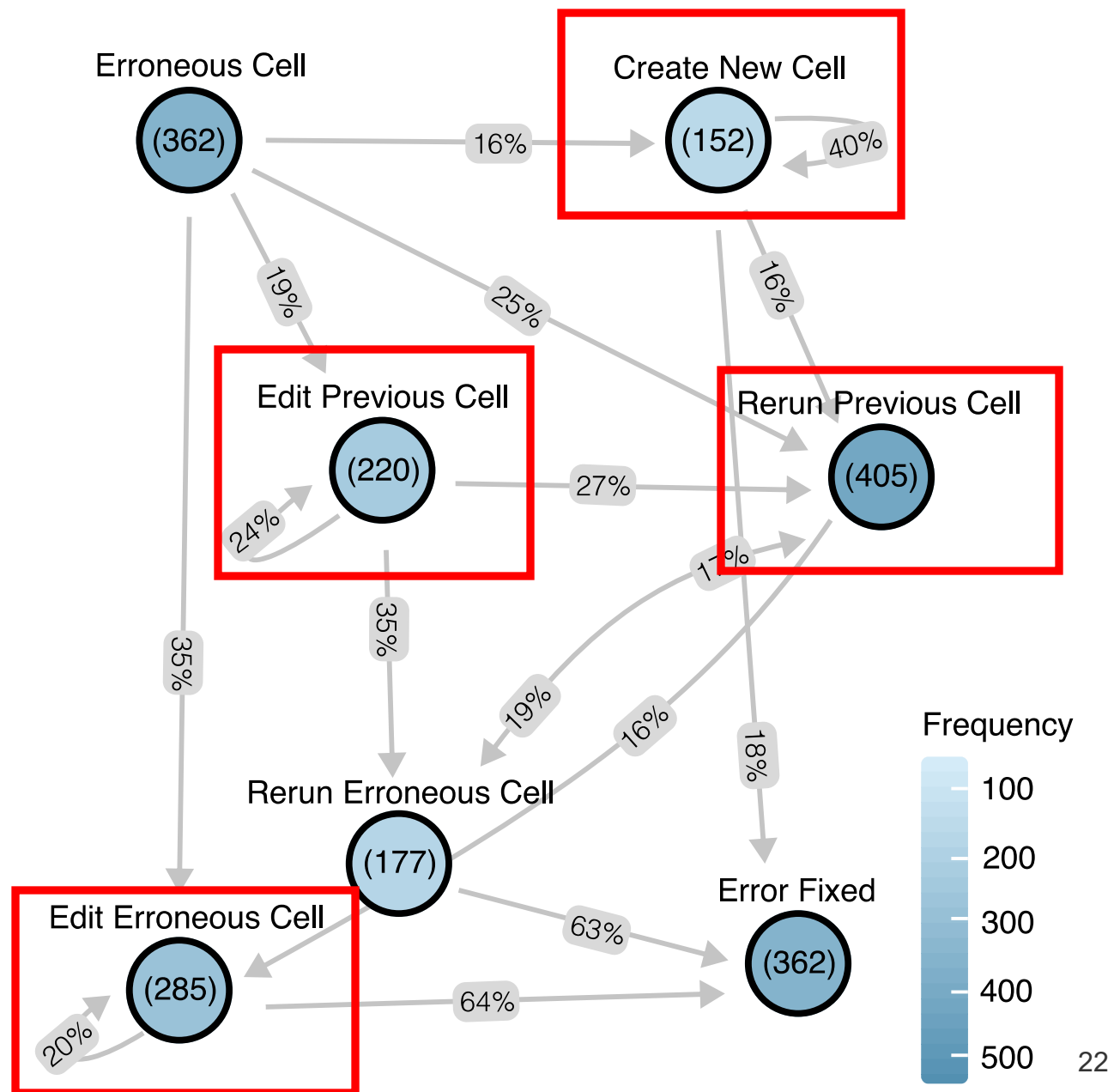


Common Programming Errors

Stage	AttrErr	TypeErr	ValErr	NameErr	NotFnd	BadReq	KeyErr	SyntaxErr	IndexErr	ModNotFnd	Misc	Total
Data Loading	9	3	0	25	28	27	6	2	1	0	0	101
Data Preprocessing	25	17	32	25	0	0	25	12	4	0	0	140
Data Exploration	24	18	37	31	0	0	20	22	14	0	4	170
Modeling	1	0	8	3	0	0	1	1	0	0	1	15
Prediction	0	1	2	1	0	0	0	1	0	0	2	7
Evaluation	2	0	0	1	0	0	0	0	0	0	0	3
Visualization	9	5	13	16	0	0	3	8	0	0	2	56
Result Saving	0	0	0	0	0	0	0	0	0	0	0	0
Comment Only	0	0	0	0	0	0	0	0	0	0	0	0
Helper Functions	0	0	19	1	0	0	0	4	0	12	1	37
Total	70	44	111	103	28	27	55	50	19	12	10	529

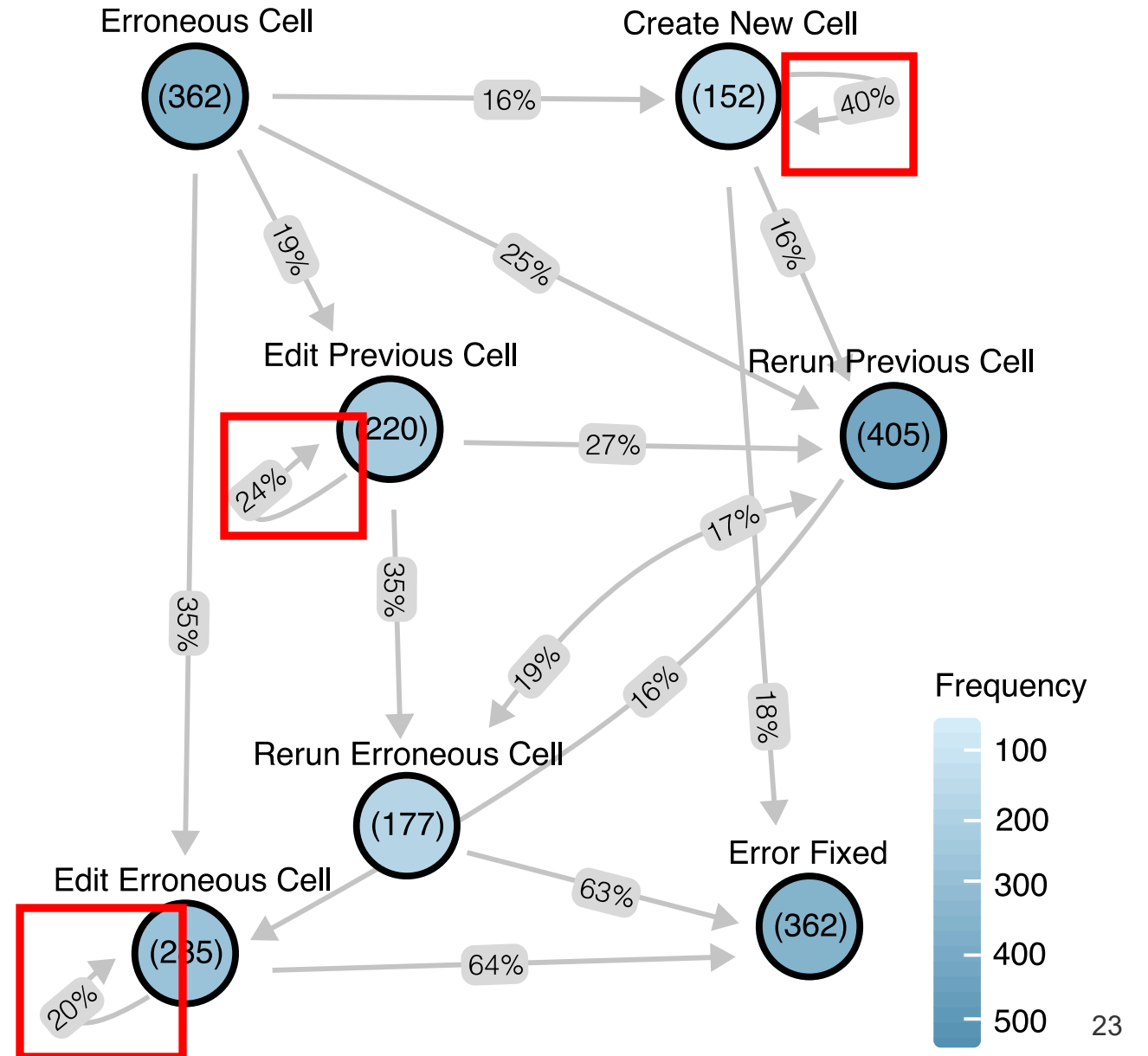


Debugging involved not only the erroneous cells.



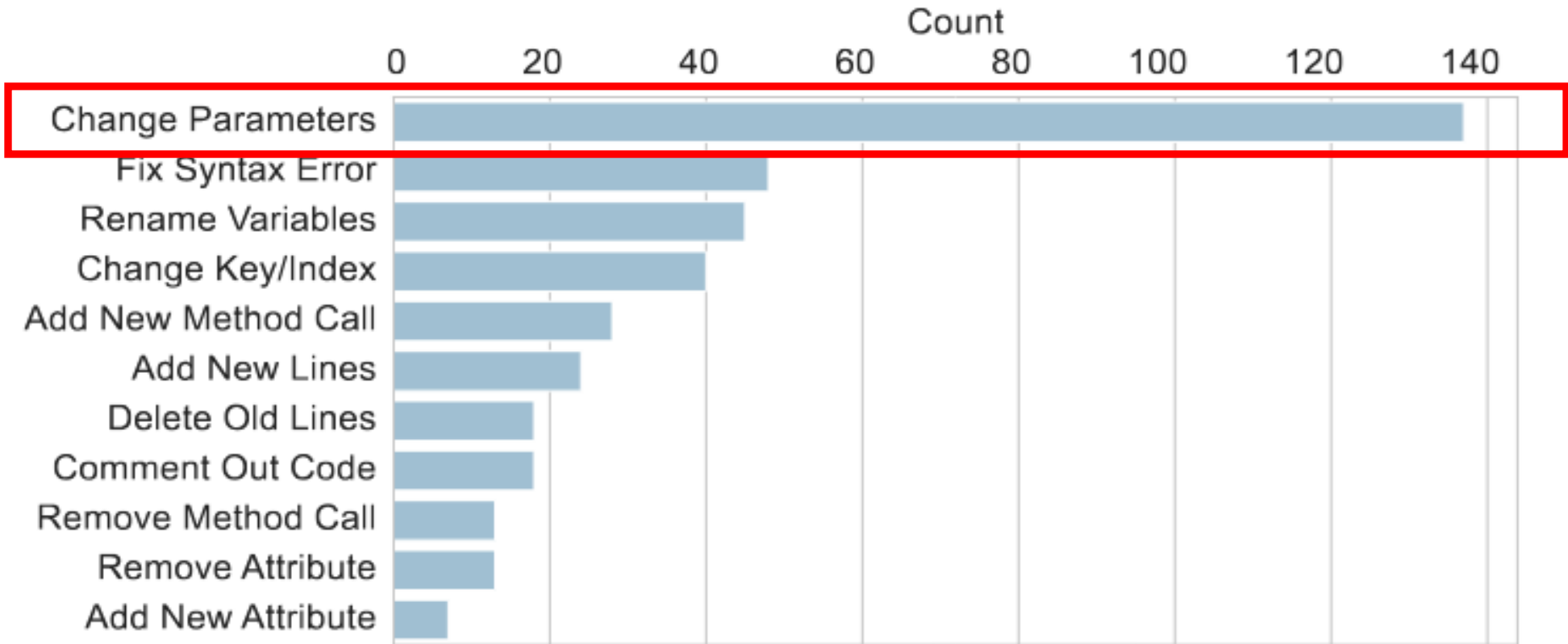


Some debugging operations were iterative.



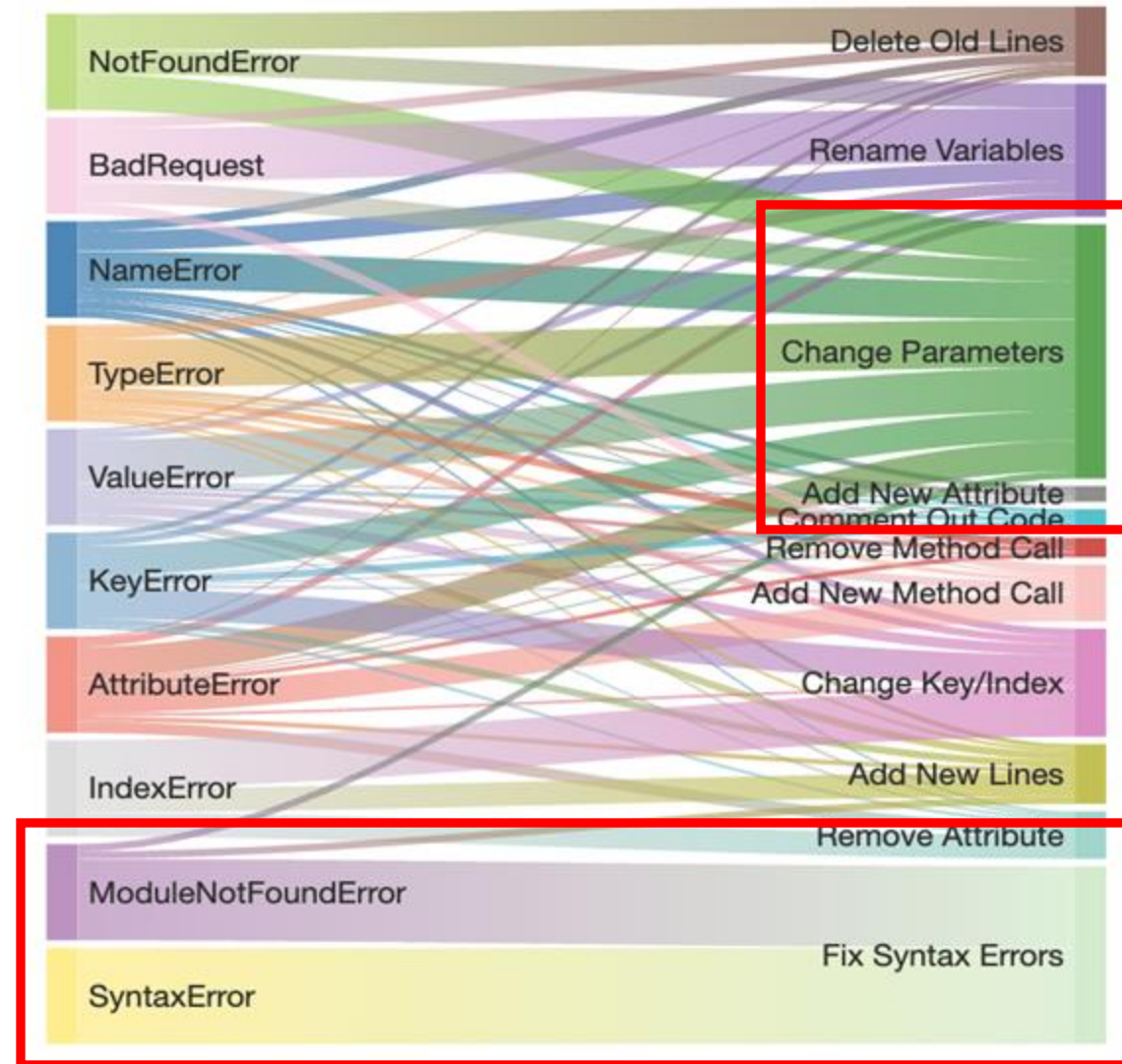


Changing parameters is the most common fixing pattern.





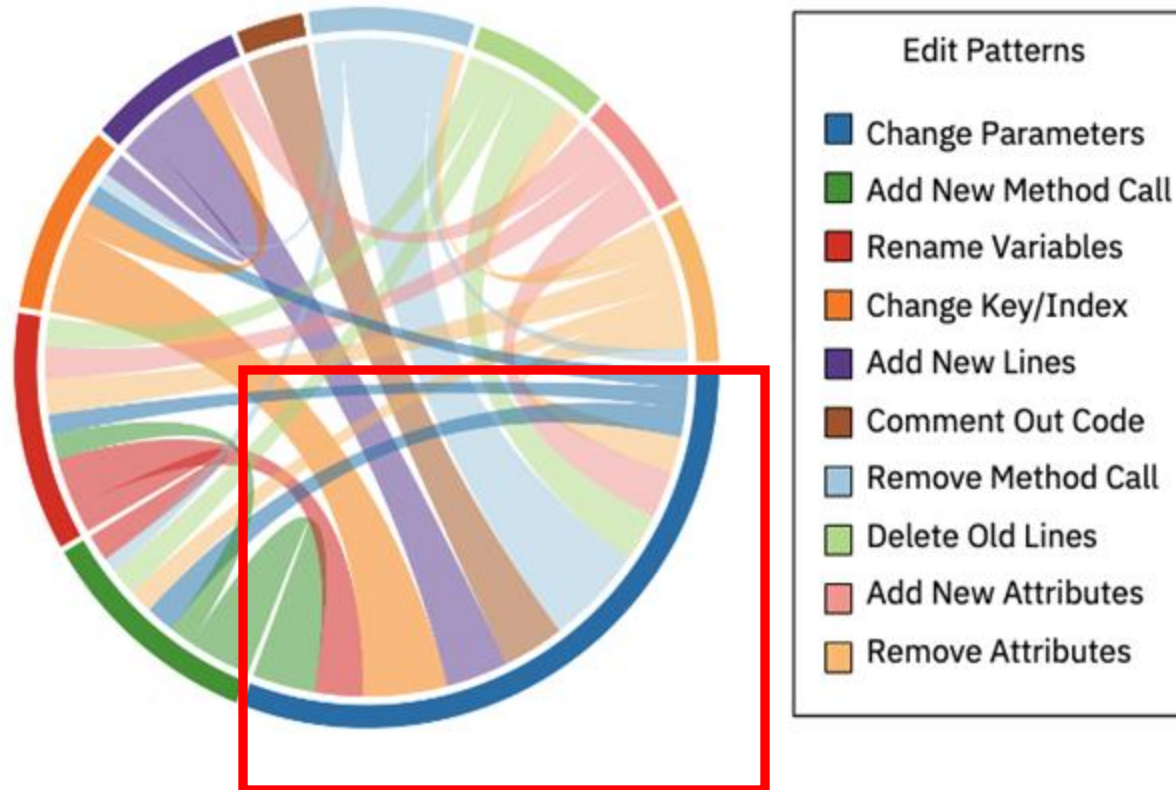
Changing parameters can fix diverse errors, while fixing syntax errors fix only simpler ones.



Sankey diagram of used fix patterns for fixing errors



Changing parameters is the most co-occurring fix pattern.



Chord diagram of co-occurring fixing patterns



Reasons for Data Preprocessing Errors

- Dirty data (9/10)
- Unclear formats (8/10)
- Lack of domain knowledge (5/10)
- Data integration (4/10)
- Large data size (4/10)
- Hard to automate cleaning (3/10)
- Complex transformation (2/10)



Reasons for Data Exploration Errors

- Dataset unfamiliarity (6/10)
- Too many columns (5/10)
- Hard to decide important features (4/10)
- Wrong assumptions (4/10)
- Inconsistent types (3/10)
- Lack of domain knowledge (3/10)



Debugging Practices

- Use print (9/10)
- Search online (6/10)
- Organize their code and test each cell (5/10)
- Execute different cells to track down errors (3/10)

Data-Centric Debugging In DS Programming

- Current debuggers focus more on identify *code logic error* not ***data errors***.
- Focus more on data context such as the characteristics of the dataset.

```
import pandas as pd

# Load dataset
df = pd.read_csv("students.csv")

# Calculate average GPA of students who slept > 6 hours
avg_gpa = df[df['sleep_hours'] > 6]['GPA'].mean()
print("Average GPA:", avg_gpa)
```



The code logic is correct

However ...



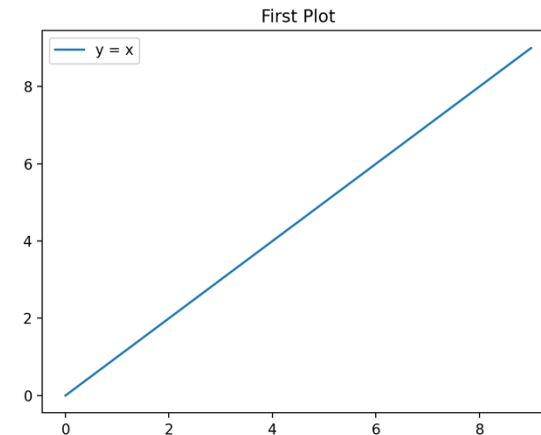
The data error is not detected

Name	Sleep Hours	GPA
Alice	8	3.5
Bob	7	3.2
Charlie	NA	3.7
David	ten	3.4
Eve	6	3.9

Supporting Iterative Debugging

- Tracing the edit history and highlighting differences in intermediate states.
- Fine-grained differential analysis on richer types of outputs, such as data tables and plots.

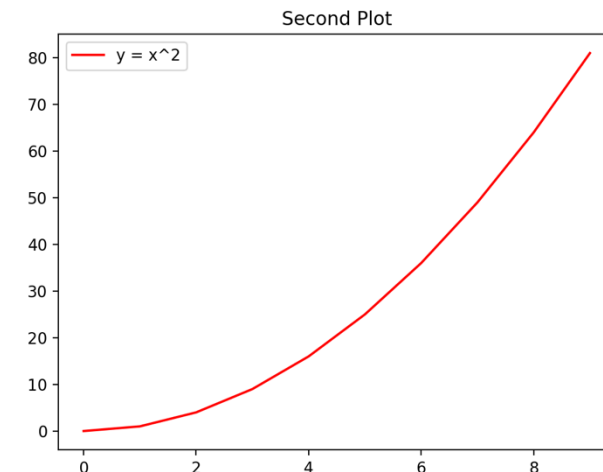
```
1 x = list(range(10))  
2 y1 = [i for i in x]  
3 plt.plot(x, y1, label='y = x')
```



Supporting Iterative Debugging

- Tracing the edit history and highlighting differences in intermediate states.
- Fine-grained differential analysis on richer types of outputs, such as data tables and plots.

```
1 x = list(range(10))  
2 y1 = [i**2 for i in x]  
3 plt.plot(x, y1, label='y = x')
```



Managing Cell Dependencies in Notebooks

- A visualization of cell dependency for programmers
- Tracking variable flows



Thank You! Takk!

Towards Understanding Fine-Grained Programming Mistakes and Fixing Patterns in Data Science

Wei-Hao Chen, Jia Lin Cheoh, Manthan Keim, Sabine Brunswicker, and Tianyi Zhang



Department of Computer Science

Scan to
find out
more →

